COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/23/26	**DATE SUBMITTED:**	07/28/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Linked Lists**

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

mainFIRST.cpp | singly_ll.h

SLL > mainFIRST.cpp > main()

```cpp
#include <iostream>
#include "singly_ll.h"

class Node{
    public:
        char data;
        Node *next;
};

//Create a simple linked list with your name, in this case "ROMAN"
int main(){
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;

    //define first node
    head->data = 'J';
    head->next = second;

    //define second node
    second->data = 'O';
    second->next = third;

    //define third node
    third->data = 'M';
    third->next = fourth;

    //define fourth node
    fourth->data = 'Z';
    fourth->next = NULL;

    // deallocation
    delete head;
    delete second;
    delete third;
    delete fourth;
    return 0;
}
```

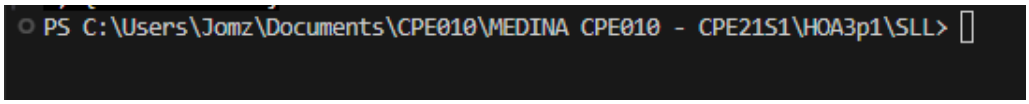
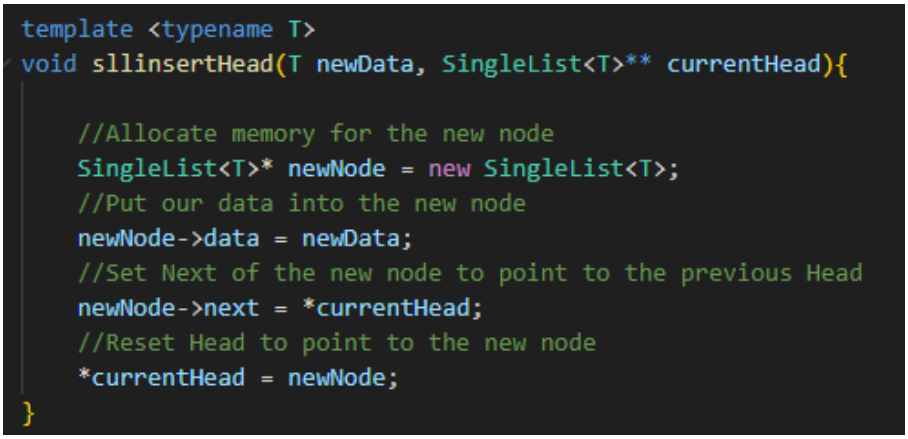
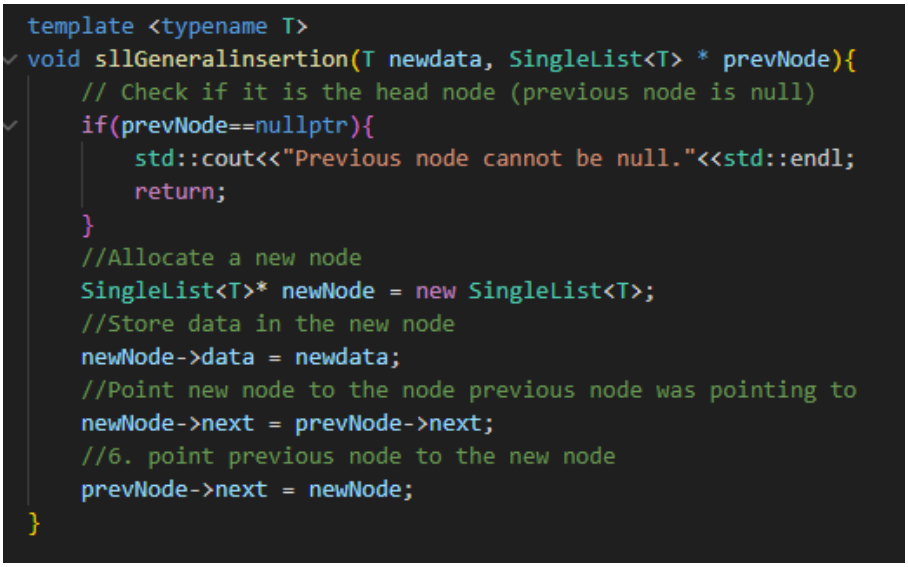
	
Discussion	The following code is as shown in the lab manual, just replacing the letters with the name. The reason why nothing shows up when the code is executed is because, although the linked list is technically created, we don't have any sort of outputting mechanism to print the data of the list. An improvement would be to create a library file to contain all of our functions for our linked list that return the values that we want, also having a way to manipulate them.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

```
11  template <typename T>
12  void listTraversal(SingleList<T>* head){
13
14      while(head != nullptr){
15          std::cout << head->data;
16
17          // to check if the next line is not equal to null
18          if (head -> next != nullptr) {
19              std::cout << "->";
20          }
21          head = head -> next;
22      }
23      std::cout << std::endl;
24  }
```

Insertion at head	
Insertion at any part of the list	

```cpp
template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead){

    //Allocate memory for the new node
    SingleList<T>* newNode = new SingleList<T>;
    //Put our data into the new node
    newNode->data = newData;
    //Set Next of the new node to point to the previous Head
    newNode->next = *currentHead;
    //Reset Head to point to the new node
    *currentHead = newNode;
}
```

```cpp
template <typename T>
void sllGeneralinsertion(T newdata, SingleList<T> * prevNode){
    // Check if it is the head node (previous node is null)
    if(prevNode==nullptr){
        std::cout<<"Previous node cannot be null."<<std::endl;
        return;
    }
    //Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //Store data in the new node
    newNode->data = newdata;
    //Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end	

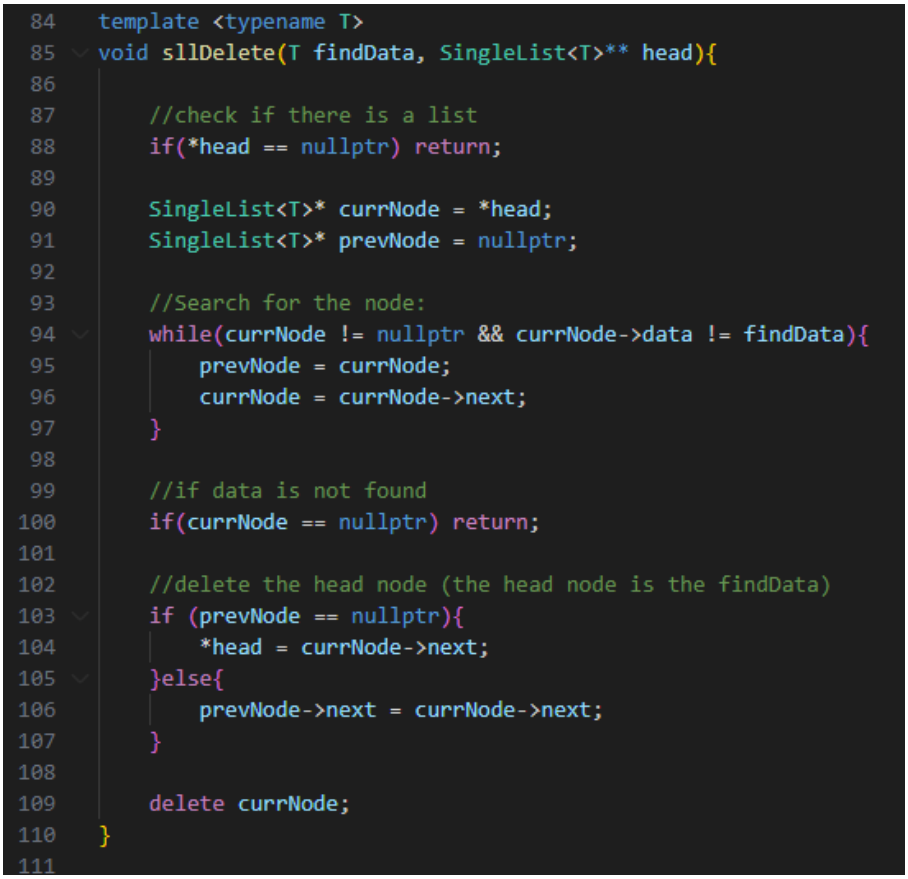
```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head) {
//Allocate new node
SingleList<T>* newNode = new SingleList<T>;

//Dereference to the head node
SingleList<T>* currentNode = *head;

//Store data in new node
newNode->data = newData;
//Point next of new node to NULL
newNode->next = nullptr;

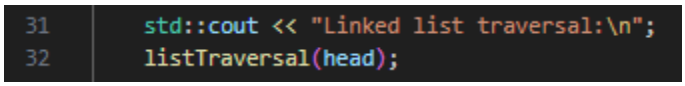
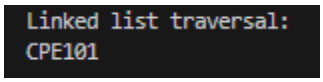
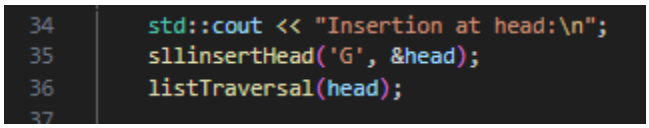
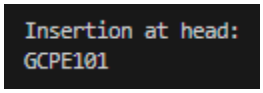
//check if the list is empty
if (currentNode == nullptr){
    currentNode == newNode;
    return;
}

//Traverse the list until next of the node is null
while(currentNode->next != nullptr){
    //iteration point to one node to another
    currentNode = currentNode->next;
}
currentNode->next = newNode;
}
```

Deletion of a node	

```
84  template <typename T>
85  void sllDelete(T findData, SingleList<T>** head){
86
87      //check if there is a list
88      if(*head == nullptr) return;
89
90      SingleList<T>* currNode = *head;
91      SingleList<T>* prevNode = nullptr;
92
93      //Search for the node:
94      while(currNode != nullptr && currNode->data != findData){
95          prevNode = currNode;
96          currNode = currNode->next;
97      }
98
99      //if data is not found
100     if(currNode == nullptr) return;
101
102     //delete the head node (the head node is the findData)
103     if (prevNode == nullptr){
104         *head = currNode->next;
105     }else{
106         prevNode->next = currNode->next;
107     }
108
109     delete currNode;
110 }
111
```

Table 3-2. Code for the List Operations

a.	**Source Code**	
	Console	
b.	**Source Code**	
	Console	

```
31      std::cout << "Linked list traversal:\n";
32      listTraversal(head);
```

```
Linked list traversal:
CPE101
```

```
34      std::cout << "Insertion at head:\n";
35      sllinsertHead('G', &head);
36      listTraversal(head);
37
```

```
Insertion at head:
GCPE101
```

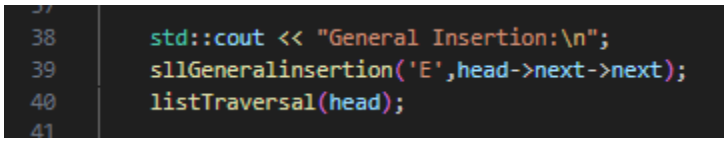
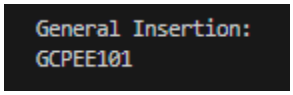
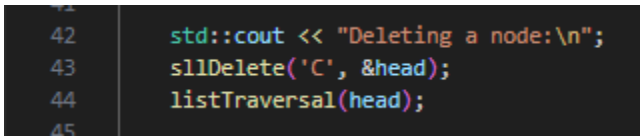
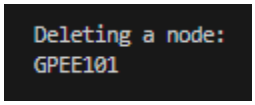
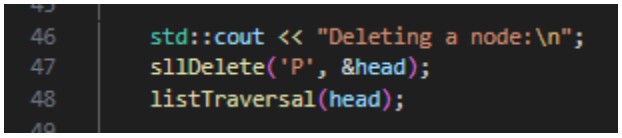
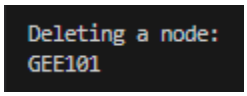
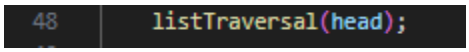
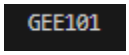
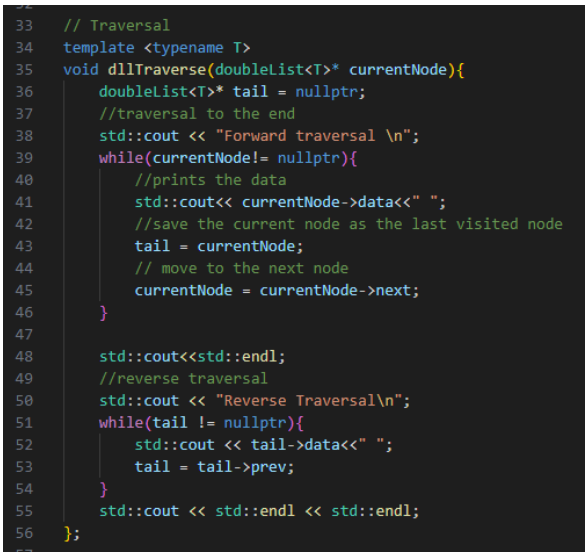
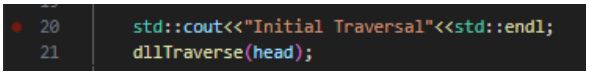
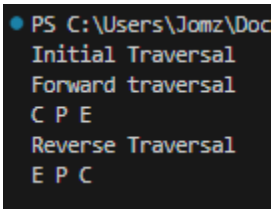
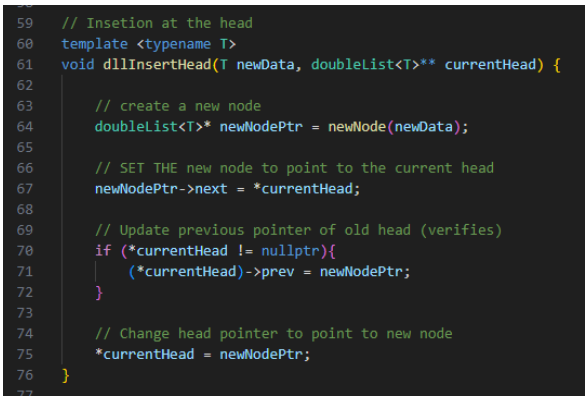
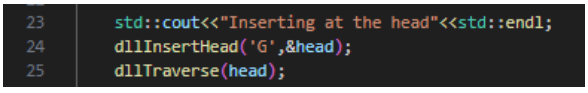
c.	**Source Code**	
	Console	
d.	**Source Code**	
	Console	
e.	**Source Code**	
	Console	
f.	**Source Code**	
	Console	

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis
  	The following code shows a template function that is designed to traverse through our created doubly linked list, forward and back, that starts from our parameter, 'currentNode'.
 	This template function, 'dllInsertHead' appends a new node to the first element of a doubly linked list, also having a check if the list is empty.

```
Inserting at the head
Forward traversal
G C P E
Reverse Traversal
E P C G
```

```
// General Insertion
template <typename T>
void dllGeneralInsertion(T newData, doubleList<T>* prevNode){
    // prev node cannot be null check
    if(prevNode==nullptr){
        std::cout << "The previous node cannot be null!\n";
        return;
    }

    // new node
    doubleList<T>* newNodePtr = newNode(newData);

    // update pointers
    newNodePtr->next = prevNode->next;
    newNodePtr->prev = prevNode;

    // middle insertion
    if(prevNode->next != nullptr){
        prevNode->next->prev = newNodePtr;
    }

    // update previous node
    prevNode->next = newNodePtr;
}
```

This template function inserts a new node after a specified existing node in our doubly linked list. It also has a check to not allow insertion at the front as the previous pointer would be at null, which may cause an error when executing.

```
    doubleList<char>* current = head;

    while(current != nullptr && current->data != 'C'){
        current = current->next;
    }

    std::cout << "General Insertion\n";
    dllGeneralInsertion('P', current);
    dllTraverse(head);
```

(Deleted C & P)

```
General Insertion
Forward traversal
C P E
Reverse Traversal
E P C
```

```cpp
// Insertion at the end
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead) {

    // create a new node
    doubleList<T>* newNodePtr = newNode(newData);

    // traverse until the last node
    while (currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    // connect the last node to the new node
    currentHead->next = newNodePtr;

    // connect the prev node to the new node
    newNodePtr->prev = currentHead;
}
```

```cpp
    std::cout<<"Inserting at the end"<<std::endl;
    dllInsertEnd('1',head);
    dllTraverse(head);
```

```
Inserting at the end
Forward traversal
G C P E 1
Reverse Traversal
1 E P C G
```

This template function appends a new node at the end of our doubly linked list, instead of our earlier one which adds it to the head. There's not much of a difference; it creates the node, checks if the next node is null (the last element), and when it does find it, it connects our new node correspondingly.

```cpp
// delete
template <typename T>
void dllDelete(T findData, doubleList<T>** head){

    // list check
    if(*head == nullptr)
        return;

    // finding specified to delete
    doubleList<T>* current = *head;
    while(current != nullptr && current->data != findData){
        current = current->next;
    }

    // if not found
    if(current == nullptr)
        return;

    // head deletion handling and else
    if(current->prev == nullptr){
        *head = current->next;

        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    } else {
        current->prev->next = current->next;

        if(current->next != nullptr){
            current->next->prev = current->prev;
        }
    }

    delete current;
}
#endif
```

```cpp
    std::cout << "Delete G\n";
    dllDelete('G', &head);
    dllTraverse(head);
```

Finally, this template function searches and removes a specific element to delete from our list, updating pointers and freeing memory accordingly. It is written to handle head, middle, and end cases.

Delete G Forward traversal C P E 1 Reverse Traversal 1 E P C	

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove

Output (modified doubly_LL.h as playlist_LL.h):

SLL > doubly > C playlist_LL.h > newNode<T>(T)

```
1   #include <iostream>
2   #ifndef PLAYLIST_LL_H
3   #define PLAYLIST_LL_H
4
5
6
7   template <typename T>
8   class playList{
9       public:
10          T data;
11          playList<T>* next = nullptr;
12          playList<T>* prev = nullptr;
13  };
14
15
16  // creating a node
17  template <typename T>
18  playList<T>* newNode(T newData){
19      //dynamically allocate memory for a new node
20      playList<T>* newNodePtr = new playList<T>;
21
22      //Store data in the node:
23      newNodePtr->data = newData;
24
25      //since its a new node, it will point to null
26      newNodePtr->next = nullptr;
27      newNodePtr->prev = nullptr;
28
29      return newNodePtr;
30  }
31
32
33  // Insertion at end of playlist (before head)
34  template <typename T>
35  void pllInsertEnd(T newData, playList<T>** currentHead) {
36
```

```
37      // create a new node
38      playList<T>* newNodePtr = newNode(newData);
39
40      // if playlist is empty
41      if(*currentHead == nullptr){
42          *currentHead = newNodePtr;
43
44          newNodePtr->next = newNodePtr;
45          newNodePtr->prev = newNodePtr;
46
47          return;
48      }
49
50      playList<T>* head = *currentHead;
51      playList<T>* tail = head->prev; // Loop back from head
52
53      // connects newNodePtr between tail and head
54      tail->next = newNodePtr;
55      newNodePtr->prev = tail;
56      newNodePtr->next = head;
57      head->prev = newNodePtr;
58  }
```

```cpp
// Insertion at head (new node is new head)
template <typename T>
void pllInsertHead(T newData, playList<T>** currentHead){

    // reuse insert end
    pllInsertEnd(newData, currentHead);

    // make new head the new head (haha)
    *currentHead = (*currentHead)->prev;
}


// General Insertion
template <typename T>
void pllGeneralInsertion(T newData, playList<T>* prevNode){
    // prev node cannot be null check
    if(prevNode==nullptr){
        std::cout << "The previous node cannot be null!\n";
        return;
    }

    // new node
    playList<T>* newNodePtr = newNode(newData);

    // update pointers
    newNodePtr->next = prevNode->next;
    newNodePtr->prev = prevNode;

    prevNode->next->prev = newNodePtr;
    prevNode->next = newNodePtr;
}

```

```cpp
// Delete
template <typename T>
void pllDelete(T findData, playList<T>** currentHead){

    // list check if empty
    if(*currentHead == nullptr)
        return;

    // finding specified to delete
    playList<T>* head = *currentHead;
    playList<T>* current = head;
    bool found = false;

    // Finding what node is needed
    do{
        if(current->data == findData){
            found = true;
            break;
        }
        current = current->next;
    } while(current != head);

    if(!found)
        return;

    // if there is only one node in the list
    if(current->next == current){
        *currentHead = nullptr;
        delete current;
        return;
    }

    // update
    current->prev->next = current->next;
    current->next->prev = current->prev;
```

```
129
130     // if head isdeleted, change head to next node
131     if(current == head){
132         *currentHead = current->next;
133     }
134
135     delete current;
136 }
137
138
139 // Traverse to next song
140 template <typename T>
141 playList<T>* pllNext(playList<T>* current){
142     if(current == nullptr)
143         return nullptr;
144     return current->next;
145 }
146
147
148 // Traverse to previous song
149 template <typename T>
150 playList<T>* pllPrev(playList<T>* current){
151     if(current == nullptr)
152         return nullptr;
153     return current->prev;
154 }
155
```

```
156
157 // Traversal (all songs)
158 template <typename T>
159 void pllTraverse(playList<T>* head){
160     if(head == nullptr){
161         std::cout << "Playlist is empty!\n";
162         return;
163     }
164
165     std::cout << "Playing songs:\n";
166     playList<T>* current = head;
167     do{
168         std::cout << current->data << "\n";
169         current = current -> next;
170     } while(current != head);
171     std::cout << "\n(back to \"" << head -> data << "\")\n\n";
172 }
173
174 #endif
```

next

```
59    // show next and previous functions
60    std::cout << "Now Playing: " << playlist->data << "\n";
61
62    playList<std::string>* nowPlaying = playlist;
63
64    nowPlaying = pllNext(nowPlaying);
65    std::cout << "Next -> " << nowPlaying->data << "\n";
66
67    nowPlaying = pllNext(nowPlaying);
68    std::cout << "Next -> " << nowPlaying->data << "\n";
69
```

```
Now Playing: Pasilyo
Next -> Hanngang Kailan
Next -> Beer
```

previous

```
70    nowPlaying = pllPrev(nowPlaying);
71    std::cout << "Previous -> " << nowPlaying->data << "\n";
72
```

```
Previous -> Hanngang Kailan
```

play all songs

```
77    // shows list loops
78    std::cout << "\nGo through the entire playlist and loop back:\n";
79    playList<std::string>* walker = playlist;
80    for(int i = 0; i < 6; i++){
81        std::cout << walker->data << " -> ";
82        walker = pllNext(walker);
83    }
84
```

```
Go through the entire playlist and loop back:
Pasilyo -> Hanngang Kailan -> Beer -> From the Start
-> Just the Way You Are -> Pasilyo ->
```

insert

```
// add song to front
std::cout << "Adding \"Pasilyo\" to the head\n";
pllInsertHead(std::string("Pasilyo"), &playlist);
pllTraverse(playlist);


std::cout << std::endl; //spacing
```

```
Adding "Pasilyo" to the head
Playing songs:
Pasilyo
Hanngang Kailan
Beer
Multo

(back to "Pasilyo")
```

```
// add to end
std::cout << "Adding \"Just the Way You Are\" to the end\n";
pllInsertEnd(std::string("Just the Way You Are"), &playlist);
pllTraverse(playlist);


std::cout << std::endl; //spacing
```

```
Adding "Just the Way You Are" to the end
Playing songs:
Pasilyo
Hanngang Kailan
Beer
Multo
Just the Way You Are

(back to "Pasilyo")
```

```
37    // insert after a specified song
38    playList<std::string>* current = playlist;
39    while(current->data != "Beer"){
40        current = current->next;
41    }
42    std::cout << "Inserting \"From the Start\" right after \"Beer\"\n";
43    pllGeneralInsertion(std::string("From the Start"), current);
44    pllTraverse(playlist);
45
46
47    std::cout << std::endl; //spacing
```

```
Inserting "From the Start" right after "Beer"
Playing songs:
Pasilyo
Hanngang Kailan
Beer
From the Start
Multo
Just the Way You Are

(back to "Pasilyo")
```

remove

```
50    // remove a song
51    std::cout << "Removing \"Multo\"\n";
52    pllDelete(std::string("Multo"), &playlist);
53    pllTraverse(playlist);
54
55
56    std::cout << std::endl; //spacing
```

```
Removing "Multo"
Playing songs:
Pasilyo
Hanngang Kailan
Beer
From the Start
Just the Way You Are

(back to "Pasilyo")
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.